

순서 기반 이벤트 전개 시스템 - 개발자를 위한 설명

WOLFPACK(ggapsang/roadmap-board 📖 설명)은 프로젝트 로드맵을 그리는 도구로 시작했다. 그런데 써 보면서 정리해 보니, 지금의 로드맵 보드는 더 일반적인 구조인 **순서 기반 이벤트 전개 시스템**의 첫 번째 활용 예라는 결론에 도달했다. ([📖 만들어진 배경](#))

문서의 목적

1. 순서 기반 이벤트 전개 시스템이 무엇인가
2. 현재 WOLFPACK이 이 시스템과 어떤 관계에 있는가
3. 앞으로 확장할 때 구조적으로 지켜야 할 방향은 무엇인가

개발자를 위한 문서이므로 프로그램 아키텍처를 어떻게 하고, 어떤 저장소나 라이브러리를 쓸지 같은 기술 선택은 다루지 않는다(그건 개발자가 더 잘 하고, 스스로 판단할 부분). 이 개념을 이해한 뒤, 이 시스템을 맡아서 확장하거나 응용하여 다른 부분에 적용하고자 했을 때, 그 대상이 이 구조와 어디서 맞고 어디서 어긋나는지를 개발자의 관점에서 설명해 주는 것이 기대하는 다음 단계다.

1. 정의

순서 기반 이벤트 전개 시스템 : 선후관계를 시간축으로 하여, 여러 트랙의 이벤트를 배치하고, 이벤트 사이의 관계를 그래프로 관리하며, 어떤 이벤트든 펼치면 그 안에 또 하나의 전개가 나타나는 구조다.

프로젝트 로드맵은 이 시스템에서 "달력 눈금을 쓰고, 트랙은 과제이며, 관계는 주로 선행인" 하나의 설정일 뿐이다.

- **순서 기반** : 축은 물리적 시간이 아니라 앞뒤 관계다. 달력은 순서 위에 얹는 눈금 중 가장 대표적인 것일 뿐이며, 날짜가 없는 이벤트도 순서만으로 온전히 배치된다.
- **이벤트** : 일정, 마일스톤, 태스크, 사고 기록, 설비 상태 변화가 모두 이벤트다. 램포트의 논리적 시간이 "사건들의 순서"로 정의되듯, 이 시스템의 시간은 이벤트들의 앞뒤 관계로 정의된다.
- **전개** : 두 가지 뜻을 동시에 가진다. 이야기나 상황이 앞뒤를 따라 펼쳐진다는 뜻(논리적 시간)과, 접힌 것을 펼친다는 뜻(부분이자 전체인 재귀 구조)이다. 카드는 접힌 이벤트이고, 보드는 펼쳐진 이벤트다.

2. 시스템의 5가지 요소

요소	역할
이벤트	앞뒤 관계를 가진 모든 단위. 접으면 카드, 펼치면 보드
트랙	이벤트가 흐르는 갈래. 펼치면 그 역시 하나의 보드
관계	이벤트 사이의 선행, 합류, 원인, 참조. 보드를 넘나듦
순서	앞뒤 관계. 필요할 때만 달력 눈금을 엮음
태스크	순서 없이 배치된 이벤트. 카드 안에 담기는 할 일(액션 아이템). 태스크끼리는 순서가 없음.

3. 핵심 원칙

3.1 순서가 먼저이다. 달력은 눈금이다

로드맵에서는 시간이 달력이지만, 프로세스 개념도에서는 날짜가 없어도 된다. 중요한 것은 “몇 월 몇일”이 아니라 “무엇이 무엇보다 먼저인가”이다.

모든 이벤트를 한 줄로 세울 필요도 없다. 서로 다른 트랙의 두 이벤트는 앞뒤를 따질 수 없는 동시적 관계일 수 있고, 관계선으로 이어질 때만 앞뒤가 정해진다. 날짜가 중요한 보드는 달력 눈금을 쓰고, 순서만 중요한 보드는 눈금 없이 순서만 쓴다. 같은 이벤트가 두 종류의 보드에 동시에 올라갈 수 있다.

3.2 모든 단위는 같은 것이다

카드, 트랙, 태스크, 보드는 서로 다른 종류가 아니라 **같은 하나의 단위**이고, 관계 속 위치에 따라 역할이 달라질 뿐이다.

- 어떤 단위를 접어 두면 카드로 보인다
- 펼쳐서 그 자식들 축 위에 늘어놓으면 보드가 된다
- 카드 안에 들어가면 태스크가 되고, 태스크가 펼쳐지면 다시 보드가 된다. 단, 같은 카드 안의 태스크끼리는 순서를 갖지 않는다.
- 트랙도 펼치면 그 안에 흐름이 하나의 보드로 드러난다.

포함은 두 방향으로 확장된다.

- **다단 중첩**: 포함 안에 포함이 들어갈 수 있고, 깊이에 제한이 없다.
- **다중 소속**: 하나의 이벤트가 여러 곳에 동시에 포함될 수 있다. a는 b에도, c에도 포함될 수 있다. 이벤트는 하나이고, b 안의 배치와 c 안의 배치가 따로 있을 뿐이다. 상태는 공유되고, 순서상 위치

와 별칭은 부모마다 다를 수 있다.

이 구조에서 포함 관계(무엇이 무엇 안에 들어 있다)와 연결 관계(무엇과 무엇이 이어지는가)가 한 그래프 안에 공존한다. 개별 객체와 그 조합을 같은 방식으로 다루는 컴포지트 패턴과 같은 원리다.

단, 포함 관계를 거슬러 올라가다 자기 자신에게 돌아오는 순환은 허용하지 않는다. 따라서 포함 관계는 부모가 하나뿐인 트리가 아니라, 부모가 여럿일 수 있되 순환이 없는 그래프(DAG)를 이룬다.

이 구조에서는 포함 관계(무엇이 무엇을 담는가)와 연결 관계(무엇이 무엇과 이어지는가)가 한 그래프 안에 공존한다. 개별 객체와 그 조합을 같은 방식으로 다루는 컴포지트 패턴과 같은 원리이며, 이론적으로는 포함과 연결을 겹쳐 정의한 밀너의 바이그래프와 닮았다. 다만 바이그래프의 포함은 트리인 반면, 이 시스템의 포함은 다중 소속을 허용한다.

3.3 태스크는 순서가 없는 포함이다

카드 안에는 하위 카드도, 태스크(액션 아이템 또는 투두리스트 같은)도 들어갈 수 있다. 둘 다 “무엇이 무엇 안에 들어 있다”는 포함 관계지만, 차이가 있다.

구분	하위 카드	태스크
담기는 곳	상위 카드	카드
서로 간의 순서	있음. 상위 카드를 펼친 보드의 순서축 위에 놓임	없음. 순서축 위에 놓이지 않음
성격	앞뒤를 따지는 이벤트의 흐름	해당 카드를 완료하기 위해 해야 할 일의 묶음

판단 규칙은 단순하다. **태스크 중 어느 둘 사이여라도 앞뒤 관계가 생기면, 앞뒤를 가진 것들은 더 이상 태스크가 아니라 하위 카드다.** 순서가 있는 것은 카드 사이의 포함 관계로 처리한다. 앞뒤가 없는 나머지는 태스크로 남겨도 되고, 같은 보드의 별도 묶음으로 올려도 된다.

예컨대 “현장 셋업” 카드가 있다면 이 카드 안에 하위 카드로 “장비 반입”, “네트워크 점검”, “담당자 교육”이 있고 이들이 어떤 순서로 해도 상관없다면 태스크다. 반대로 “장비 반입이 끝나야 네트워크 점검을 할 수 있다”는 앞뒤가 생기면, 이들은 “현장 셋업”을 펼친 보드 위의 하위 카드가 되고, 그 사이에 선행 관계가 그어진다.

이때 하위 카드 전체가 한 줄로 설 필요는 없다. “현장 셋업”을 펼친 보드가 “장비 반입 → 네트워크 점검 → 시스템 연동 시험”이라는 순서의 하위 카드들과, “담당자 교육 → 평가”라는 하위 카드로 나누어질 수 있다. 이러한 묶음 안에서는 앞뒤가 정해지지만, 두 묶음 사이에는 관계선이 없으므로 순서가 없다. 3.1에서 말했듯이, 순서는 관계선으로 이어진 것들 사이에만 생긴다는 원칙이 그대로 적용된다. 이런 묶음은 펼친 보드에서 각각 하나의 트랙으로 놓는 것이 자연스러우며, 트랙끼리 순서가

없는 것과 같은 모습이 된다. 하위 카드의 순서는 모두가 한 줄로 서는 전순서가 아니라, 일부만 앞뒤가 정해지는 부분순서다.

따라서 태스크 목록이 화면에 나열된다면 순서는 보기 편의를 위한 표시일 뿐 선후 관계의 의미를 갖지 않는다. 사용자가 목록을 끌어서 재배치하더라도 그것은 앞뒤 관계가 아니다.

태스크와 이벤트는 단위의 종류가 다른 것이 아니라 **배치 방식이 다른 것이다**. 같은 단위가 카드 안에서 순서 없이 담기면 태스크로, 보드의 순서축 위에 놓이면 카드로 보인다. 따라서 다음이 모두 가능해야 한다.

- 하위 카드 사이의 순서가 의미 없어지면, 순서축에서 내려 상위 카드의 태스크로 담는다.
- 태스크 사이에 앞뒤가 생기면, 하위 카드로 올린다.
- 같은 단위가 한 보드에서는 어떤 카드의 태스크로, 다른 보드에서는 날짜를 가진 카드로 동시에 나타난다. 이벤트와 배치가 분리되어 있으므로 상태 변화는 양쪽에 함께 반영한다.

3.4 이벤트는 보드 밖에 존재하고, 보드는 배치만 가진다

한 보드의 카드가 다른 보드의 카드와 관계를 갖는 일은 자연스럽게 생긴다. 사고 타임라인의 원인 카드는 로드맵의 개선 과제 카드와 이어지고, 인허가 보드의 승인 시점은 로드맵의 현장 설치 일정의 선행 이벤트가 된다.

이렇게 되려면 이벤트가 보드 안에 속해 있어서는 안 된다. 이벤트 보드 밖에 독립적으로 존재하고, 보드는 “이 이벤트를 이 트랙의 이 위치에 놓는다”는 배치 정보만 가진다. 그러면 같은 이벤트가 여러 보드에 동시에 나타나고, 한쪽에서 상태를 바꾸면 모든 보드에 반영된다.

3.5 본질과 표시를 나눈다

같은 이벤트가 여러 보드에 올라가므로, 무엇이 이벤트의 본질이고 무엇이 보드에서의 표시인지를 나눠야 한다.

속하는 곳	항목	이유
이벤트 (본질)	제목, 상태, 담당, 실제 날짜	어느 보드에서 보든 같아야 함
배치 (표시)	트랙, 순서상 위치, 표시 방식, 별칭	보드마다 다를 수 있음

별칭은 같은 이벤트 보드의 맥락에 맞는 다른 이름으로 부를 수 있게 한다. 예를 들어 로드맵에서 “3D 모델링”인 이벤트가 3D 모델러의 커리어 보드에서는 “물류 로봇 3D 모델링 경험”으로 보일 수 있다. 이름은 달라도 같은 이벤트이므로 상태 변화는 공유된다.

3.6 관계는 독립된 대상이며, 종류마다 규칙이 다르다

관계는 이벤트에 딸린 부속 정보가 아니라 이벤트와 동등한 독립 대상이다. 보드와 무관하게 존재하고, 보드를 넘나든다. 한 이벤트가 지연되면 관계를 따라 다른 보드의 연결 이벤트까지 영향이 퍼지는 것도 이 덕분에 표시할 수 있다.

관계는 종류를 가지며, 종류마다 구조적 성질이 다르다.

관계 종류	의미	구조적 성질
선행	A가 끝나야 B가 시작	순환이 생기면 안 됨
합류	여러 갈래가 하나로 모임	한 이벤트는 하나의 합류 대상만 가짐 (트리)
원인	A가 B를 일으킴	순환이 생기면 안 됨
참조	단순히 관련 있음	순환 허용
포함	A 안에 하위 카드 B가 들어 있음 (펼침)	다단 중첩과 다중 소속 허용, 순환은 안 됨 (DAG). 하위 카드끼리는 순서를 가짐
태스크 소속	카드 A 안에 할 일 B가 들어 있음	다중 소속 허용. 태스크끼리는 순서가 없음. 순서가 생기면 포함으로 전환

관계의 종류는 앞으로 용도가 늘어나면서 함께 늘어난다.

4. 용도가 달라져도 구조는 같다

여러 용도로 쓸 수 있지만, 차이를 만드는 변수는 몇 개 없다. 세로축이 달력인지 순서인지, 그리고 트랙과 카드와 연결선이 무엇을 의미하는지가 차이를 만든다.

용도	트랙의 의미	카드	연결선의 의미	세로축
프로젝트의 모임으로서 특정 업무	개별 프로젝트	개별 프로젝트의 주요 단계	계승	달력
프로젝트 관리	과제·조직	프로젝트 내부의 단계	트랙 간 인계, 선행 조건	달력
프로세스 수렴 트리	갈래	시점	합류 (여럿 → 하나)	순서

사고 타임라인	사람·설비·관리시스템	시점	원인-결과	달력 (분·초)
---------	-------------	----	-------	----------

용도마다 별도의 도구를 만들 필요가 없다. 이 변수들을 설정으로 바꿀 수 있는 하나의 구조면 충분하다. 계획 대비 실적, 리스크 트랙, 의사결정 게이트, 과부하 구간 표시 같은 로드맵의 심화 기능도 새로운 구조가 아니라 이 변수와 관계 종류를 늘리는 것으로 설명된다.

5. WOLFPACK과의 관계

5.1 울프팩은 이 시스템의 한 가지 설정이다

현재 로드맵 드로잉 툴인 WOLFPACK의 구성 요소는 이 순서 기반 이벤트 전개 시스템의 개념에 그대로 대응한다.

현재의 WOLFPACK	시스템 개념
로드맵 보드 하나	펼쳐진 이벤트 하나 (프로젝트라는 이벤트를 펼친 상태)
과제별 트랙(열)	트랙
일정 카드	기간을 가진 이벤트
마일스톤	시점 이벤트
카드 사이의 연결선	관계 (선행)
세로 달력축	달력 눈금을 엮은 순서축

즉 현재 프로그램은

- 보드가 하나이며
- 축에 항상 달력 눈금이 있고
- 관계의 종류는 선행 하나이며
- 이벤트는 한 보드에만 존재하고
- 이벤트를 펼쳐 하위 보드로 돌아가는 동작이 없는

위와 같은 조건이 고정된 상태의 순서 기반 이벤트 전개 시스템이다.

이 조건들은 지금 단계에서는 전혀 문제가 아니다. 다만 이것이 시스템의 본질이 아니라 현재의 설정 값이란느 점을 구현해서 구분에 두는 것이 중요하다.

6. 확장을 위한 제언

이 제언들은 구현 방법이 아니라 **구조적으로 지켜야 할 방향**이다. 당장 기능으로 드러나지 않아도, 구조가 이 방향을 막지 않으면 된다.

1. 보드가 하나여도, 이벤트와 배치는 나눠서 생각한다

보드가 하나일때는 둘을 구분할 실익이 없어 보인다. 하지만 이 구분은 나중에 추가하기 가장 어려운 부분이다. 같은 이벤트가 두 보드에 올라가는 첫 번째 순간, 구분이 없는 구조는 전체를 다시 짜야 한다.

2. 이벤트 보드는 넘어서도 식별될 수 있어야 한다

모든 이벤트에 보드와 무관하게 유일한 식별 수단을 둔다. 보드 안에서의 순번처럼 보드에 의존하는 식별은, 보드를 넘나드는 관계가 생긴 순간 충돌한다

3. 관계는 이벤트와 동등한 독립 대상으로 다룬다

연결선을 카드의 부속 정보로 두면, 보드를 넘나드는 관계와 관계의 종류를 표현할 수 없다. 관계는 따로 존재하고, 종류를 가지며, 그 종류의 목록은 코드에 박지 않고 설정으로 늘릴 수 있어야 한다.

4. 시간은 순서가 기본이고, 달력은 선택적으로 없는 눈금이다

화면 배치 로직 날짜에 묶여 있지 않는다. 날짜 없는 보드도 만들 수 있어야 한다. 위치를 계산할 때는 “순서상 위치”를 먼저 구하고, 달력 눈금이 있을 때만 그것을 날짜로 환산하는 구조면 두 종류의 보드가 같은 코드 위에서 동작한다. 축의 방향(세로/가로) 역시 데이터가 아니라 화면 표시의 선택으로 둘 수도 있다.

5. 본질과 표시의 경계를 지킨다

새 속성을 추가할 때마다 “이것은 어느 보드에서 보든 같아야 하는가, 보드마다 달라도 되는가”를 묻는다. 판단이 애매하면 표시 쪽에 둔다. 표시 쪽에 두었던 것을 나중에 본질로 올리는 것은 쉽지만, 반대는 데이터를 쪼개는 작업이 된다.

6. 이벤트가 자식을 가질 수 있는 자리를 남긴다

카드를 펼쳐 보드가 되는 기능, 카드 안에 태스크가 들어가는 기능은 당장 필요하지 않다. 하지만 이벤트가 다른 이벤트를 품을 수 있다는 가능성을 구조가 막지 않아야 한다. 카드, 태스크, 보드를 서로 다른 종류로 굳혀 버리면 이 가능성이 닫힌다.

이 때 두가지 포함을 구분해 둔다. 하위 카드는 순서를 가진 포함이고, 태스크는 순서가 없는 포함이다. 태스크에 앞뒤가 생기는 순간 하위 카드로 전환되어야 하므로, 태스크와 하위 카드는 같은 단위이고, “순서축 위에 놓이는가”라는 배치 방식만 다르게 다뤄져야 한다.

7. 관계 종류별 구조 규칙을 의식한다

선행과 원인은 순환이 생기면 안 되고, 합류는 트리를 이뤄야 하며, 참조는 자유롭다. 지금은 선행 하나뿐이지만, 관계를 저장하고 검증하는 지점을 한 곳으로 모아 두면 종류가 늘어나도 규칙을 추가하기 쉽다.

8. 보이지 않는 것은 막지만 않으면 된다

여러 보드 전환, 순서형 축, 펼침과 드릴다운, 보드 간 영향 전파, 보드별 공유 범위 같은 기능은 지금 구현하지 않아도 된다. 단, 요구는 하나다. **지금 내리는 구조적 결정이 이것들을 불가능하게 만들지 않을 것.**

판단 기준

기능을 추가하거나 구조를 결정할 때 다음 질문 하나로 대부분 판단할 수 있다

이것은 이 보드에서만 성립하는가, 아니면 이벤트 자체에 성립하는가?

보드에서만 성립하는 것은 배치와 표시로, 이벤트 자체에 성립하는 것은 이벤트와 관계로 보낸다.

7. 이 시스템의 역할

드로잉 툴 자체만으로는 여러 기능을 추가한다 하더라도 가치가 있는 것은 아니다. 대체제들이 있고 (파워포인트, 노션, 지라 등) 더 나은 도구를 쓰기 위해 새 도구를 익히려는 수고를 하지 않으려는 사람이 더 많다.

다만 데이터와 업무에 시스템이 연결될 때 다음과 같은 것들이 가능해 질 수 있다.

- **특정 도메인 데이터에 직접 연결될 때:** 설비 로그나 센서 이벤트가 자동으로 흘러 들어와 이상 이벤트를 만드는 타임라인이 그려지고, 원인 카드가 다음 조치 액션 카드로 이어진다면, 그것은 드로잉 툴이 아니라 분석·보고 업무를 대신하는 도구가 된다.
- **기존 B2B 솔루션의 한 모듈이 될 때:** 기존 솔루션 안에서 "이벤트를 순서와 관계로 보여주는 뷰"로 들어갈 수 있다.
- **보고·감사 산출물이 규제나 계약으로 요구될 때:** 인허가, 사고 조사, 프로젝트 감리처럼 "이 흐름을 문서로 제출해야 하는" 영역처럼 그리는 수고 자체가 비용인 경우에 쉽게 쓰일 수 있다.

8. UI 철학: 하나의 문법

이 시스템의 UI는 하나의 조작과 하나의 문법으로 이루어진다.

조작은 **접기와 펼치기**뿐이다. 접으면 카드, 펼치면 보드가 된다. 프로젝트를 펼치면 과제들의 보드가, 과제를 펼치면 단계들의 보드가, 단계를 펼치면 다시 그 안의 흐름이 나타난다.

문법은 **순서축, 트랙, 관계선** 세 가지다. 어느 층위로 펼쳐 들어가도 이 문법은 바뀌지 않는다. 위아래로는 순서가 흐르고, 옆으로는 갈래가 나란히 서고, 그 사이를 관계선이 잇는다.

재귀 구조의 가장 큰 약점은 사용자가 지금 어느 층위에 있는지 잃어버린다는 것이다. 이 시스템은 층위마다 다른 화면을 만드는 대신, 모든 층위에 같은 화면 규칙을 적용하는 방식을 적용한다. 사용자는 새로운 화면을 배울 필요 없이, 이미 아는 문법을 한 단계 안쪽에서 다시 읽으면 된다.

축을 순서로 두고 달력을 선택적 눈금으로 엮는 것도 같은 철학에서 나온다. 날짜가 있는 로드맵과 날짜가 없는 프로세스 개념도가 서로 다른 도구가 아니라, 눈금만 다른 같은 화면이 된다.

제품보다 설계 언어로 남고, UI 컨셉의 진전을 위해 이 글을 정리해 둔다.